

[Marcar como hecha](#)

12.2. Generación dinámica de páginas. Objetos implícitos

Procesamiento de la página JSP

Cuando se llame a ésta página (date.jsp), será compilada (por el motor JSP) en un Servlet Java. En este momento el Servlet es manejado por el motor Servlet como cualquier otro Servlet. El motor Servlet carga la clase Servlet (usando un cargador de clases) y lo ejecuta para crear HTML **dinámico** para enviarlo al navegador. Para este ejemplo, el Servlet crea un objeto Date y lo escribe como un String en el objeto out, que es el stream de salida hacia el navegador.

```
<h1>Hola, mundo!</h1>
<p> Hoy es <%= new java.util.Date() %> </p>
```

Objetos implícitos

El motor JSP nos ofrece instancias de un conjunto de clases. Son objetos ya establecidos, que no tenemos más que usar (no hay que instanciarlos). Deben utilizarse dentro del código Java.

Algunos objetos implícitos:

page (javax.servlet.jsp.HttpJspPage.): Instancia del servlet de la página. Esto es sólo un sinónimo de this, y no es muy útil en Java. Fue creado como situación para el día que los lenguajes de script puedan incluir otros lenguajes distintos de Java.

config (javax.servlet.ServletConfig): Datos de configuración del servlet.

request (javax.servlet.http.HttpServletRequest): Datos de la petición, incluyendo los parámetros. Este es el HttpServletRequest asociado con la petición, y nos permite mirar los parámetros de la petición (mediante **getParameter**), el tipo de petición (GET, POST, HEAD, etc.), y las cabeceras HTTP entrantes (cookies, Referer, etc.). Estrictamente hablando, se permite que la petición sea una subclase de ServletRequest distinta de HttpServletRequest, si el protocolo de la petición es distinto del HTTP. Esto casi nunca se lleva a la práctica.

response (javax.servlet.http.HttpServletResponse): Datos de la respuesta. Este es el HttpServletResponse asociado con la respuesta al cliente. Como el stream de salida tiene un buffer, es legal seleccionar los códigos de estado y cabeceras de respuesta, aunque no está permitido en los servlets normales una vez que la salida ha sido enviada al cliente.

out (javax.servlet.jsp.JspWriter): Flujo de salida para el contenido de la página. Este es el PrintWriter usado para enviar la salida al cliente. Sin embargo, para poder hacer útil el objeto response esta es una versión con buffer de PrintWriter llamada JspWriter. Podemos ajustar el tamaño del buffer, o incluso desactivar el buffer, usando el atributo buffer de la directiva page. Se usa casi exclusivamente en scriptlets ya que las expresiones JSP obtienen un lugar en el stream de salida, y por eso raramente se refieren explícitamente a out.

session (javax.servlet.http.HttpSession): Datos específicos de la sesión de un usuario. Este es el objeto HttpSession asociado con la petición. Las sesiones se crean automáticamente, por esto esta variable se une incluso si no hubiera una sesión de referencia entrante. La única excepción es usar el atributo session de la directiva page para desactivar las sesiones, en cuyo caso los intentos de referenciar la variable session causarán un error en el momento de traducir la página JSP a un servlet.

application (javax.servlet.ServletContext): Datos compartidos por todas las páginas de una aplicación. El ServletContext obtenido mediante getServletConfig().getContext().

pageContext (javax.servlet.jsp.PageContext): Datos de contexto para la ejecución de la página. JSP presenta una nueva clase llamada PageContext para encapsular características de uso específicas del servidor como JspWriters de alto rendimiento. La idea es que, si tenemos acceso a ellas a través de esta clase en vez directamente, nuestro código seguirá funcionando en motores servlet/JSP "normales".

?

exception (java.lang.Throwable): Errores o excepciones no capturadas.

Ejemplo:

```
<%  
String strParam = request.getParameter("nombre_del_parametro");  
out.println( strParam );  
%>
```

Los objetos implícitos más utilizados definidos por Tomcat son out y request, seguidos de application y session. Pero los repasé en orden alfabético para facilitar la referencia.

Tanto si creas objetos dentro de las páginas JSP como si

Tomcat los crea implícitamente por ti, no puedes usarlos adecuadamente a menos que sepas en qué ámbito están disponibles.

Hay cuatro ámbitos posibles. En orden de generalidad creciente, son page, request, sesión y aplicación. Aprenderás más sobre ellos en las siguientes secciones.

En general, si no está seguro de la clase a la que pertenece un objeto determinado, puede siempre puede mostrar su nombre con la siguiente expresión:

```
<%=el_objeto_misterioso.getClass().getName()%>
```

Última modificación: martes, 14 de marzo de 2023, 13:14